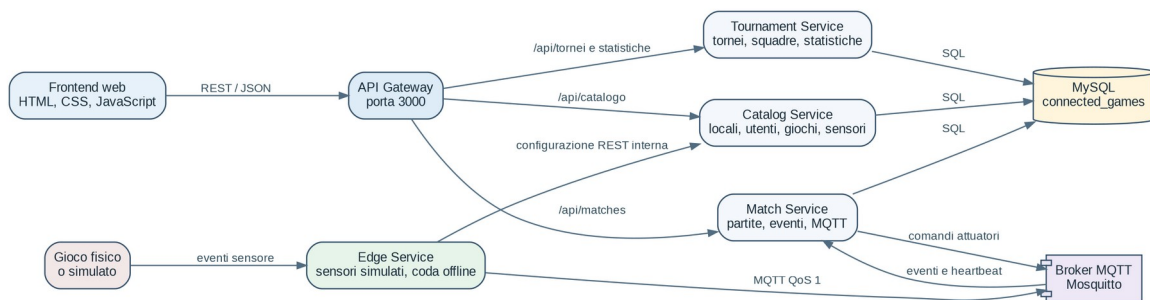


# PlayConnect

## Connected Games Platform

Progetto di Laboratorio PISSIR  
Anno Accademico 2025/2026

### Gruppo PlayConnect



### Vista generale della piattaforma

## Indice

1. 1. Introduzione
2. 2. Specifiche funzionali
3. 3. Analisi tecnologica
4. 4. Scelta dell approccio
5. 5. Architettura del software
6. 6. Modello del dominio e database
7. 7. API REST e MQTT
8. 8. Implementazione
9. 9. Interfaccia utente
10. 10. Validazione
11. 11. Procedura di demo
12. 12. Limiti e sviluppi futuri

## 1. Introduzione

PlayConnect è una piattaforma software che collega giochi tradizionali fisici a Internet. Il gioco continua a essere usato normalmente, mentre sensori o pulsanti registrano eventi come inizio, punto, tiro e fine della partita. Un dispositivo edge raccoglie questi eventi e li invia alla piattaforma centrale.

La piattaforma conserva partite, risultati, tornei e statistiche. Il progetto usa sensori simulati, ma il formato dei messaggi può essere usato anche da Arduino, ESP32 o Raspberry Pi.

## 2. Specifiche funzionali

Sono previsti quattro ruoli con funzioni diverse.

| Ruolo                      | Funzioni                                                                        |
|----------------------------|---------------------------------------------------------------------------------|
| Giocatore                  | Visualizza giochi, partite, statistiche personali, tornei e classifiche.        |
| Amministratore locale      | Gestisce giochi, client, edge, sensori, attuatori e partite del proprio locale. |
| Amministratore gioco       | Definisce tipi di gioco, eventi, limite e modelli dei sensori.                  |
| Amministratore piattaforma | Gestisce locali, amministratori, tornei e statistiche globali.                  |

Le principali funzioni implementate sono:

- gestione dei locali e dei giochi installati;
- tipi di gioco configurabili senza modificare il Match Service;
- partite individuali e a squadre;
- eventi con UUID e valore numerico;
- funzionamento edge online e offline;
- tornei su uno o più locali;
- statistiche globali, locali e personali;
- interfaccia web per tutti i ruoli.

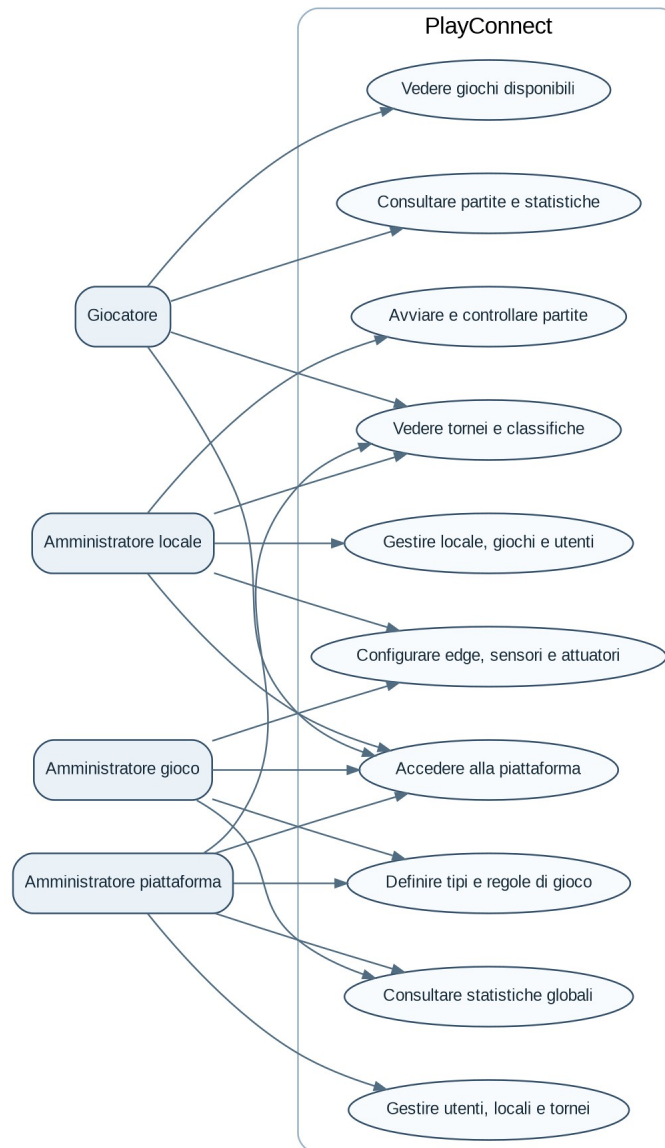


Figura 1 - Diagramma dei casi d'uso

### 3. Analisi tecnologica

Per il controllo dei giochi si poteva usare soltanto REST, ma in questo caso ogni sensore avrebbe dovuto conoscere direttamente il server. MQTT è più adatto agli eventi piccoli e frequenti, permette la consegna asincrona e separa il dispositivo dal servizio che elabora i dati.

| Tecnologia            | Uso nel progetto                               |
|-----------------------|------------------------------------------------|
| Node.js ed Express    | API Gateway, microservizi ed Edge Service.     |
| MySQL 8               | Dati centrali e relazioni tra entità.          |
| Mosquitto MQTT        | Eventi sensore, heartbeat e comandi attuatori. |
| HTML, CSS, JavaScript | Interfaccia web senza framework.               |
| Docker Compose        | Avvio ripetibile di tutti i componenti.        |
| OpenAPI 3.0.3         | Documentazione delle API REST.                 |



## 5.2 Deployment

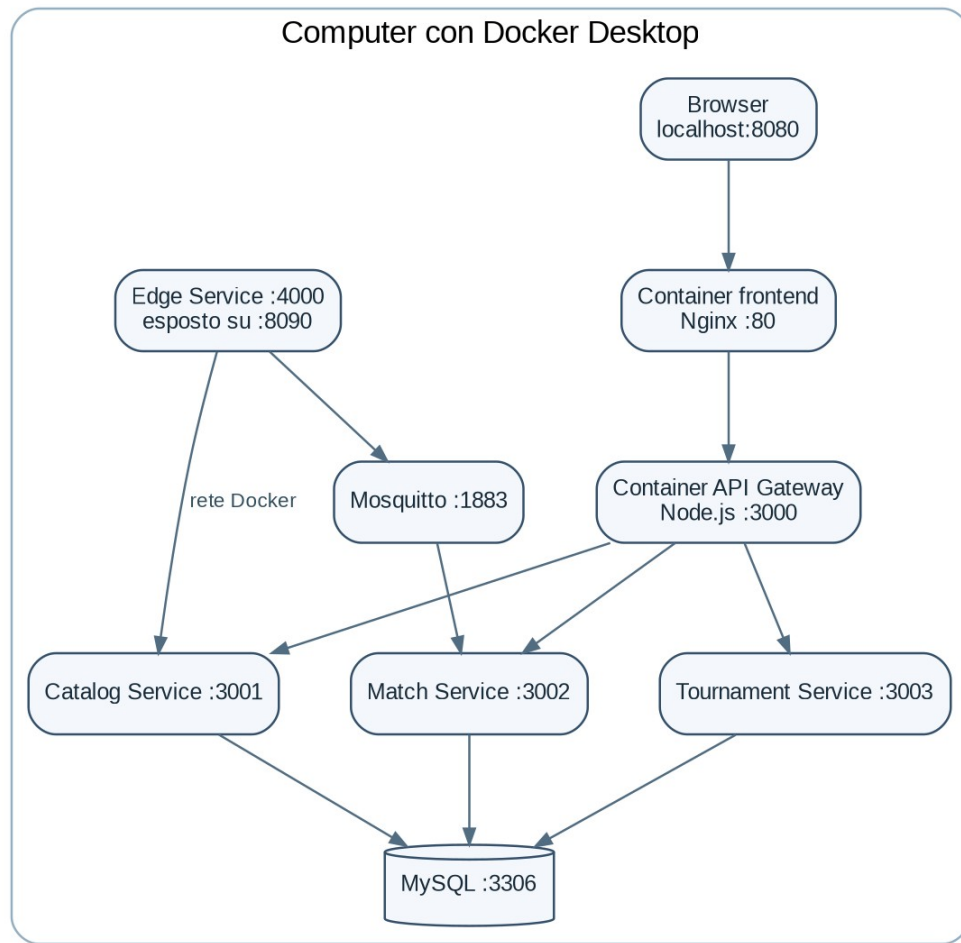


Figura 4 - Deployment con Docker Compose

## 6. Modello del dominio e database

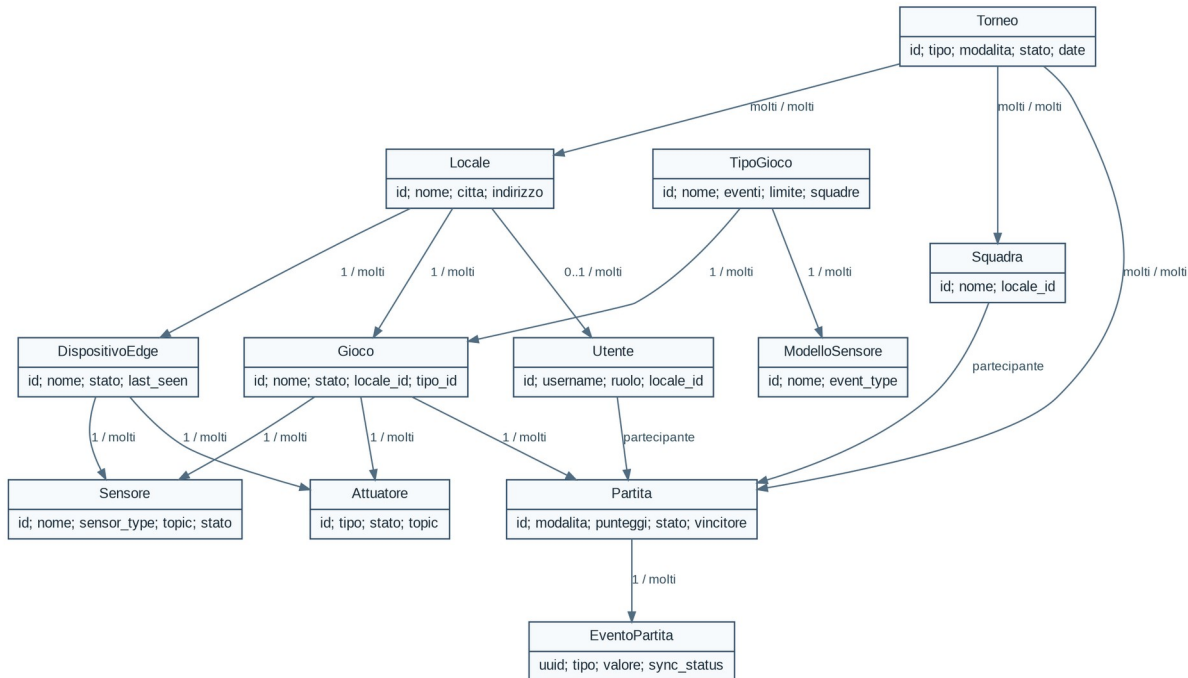


Figura 5 - Classi del dominio

La tabella `game_types` contiene le regole del gioco: evento di inizio, evento punto 1, evento punto 2, evento di fine, limite di punteggio e supporto alle squadre. La tabella `match_events` salva UUID, valore, stato di sincronizzazione e dati essenziali della sorgente.

| Tabella                                 | Contenuto                                    |
|-----------------------------------------|----------------------------------------------|
| locales, users                          | Locali e utenti con ruolo.                   |
| game_types, sensor_templates            | Regole e modelli dei sensori.                |
| games, edge_devices, sensors, actuators | Installazione fisica o simulata.             |
| matches, match_events                   | Partite ed eventi ricevuti.                  |
| teams, team_members                     | Squadre e componenti.                        |
| tournaments e tabelle ponte             | Tornei, locali, squadre e partite collegate. |

## 7. API REST e MQTT

L API pubblica contiene 41 percorsi ed è descritta in `docs/openapi.yaml`.

```

locales/{localeId}/games/{gameId}/matches/{matchId}/events
locales/{localeId}/edge/{deviceId}/heartbeat
locales/{localeId}/games/{gameId}/actuators/{actuatorId}/commands

```

Esempio di evento delle freccette:

```

{
  "event_uuid": "edge-1-20-1720000000-3",
  "event_type": "DART_THROW_PLAYER_1",
  "value": 60,
  "sync_status": "SYNCED"
}

```

## 8. Implementazione

### 8.1 Regole configurabili

Il file `gameRules.js` trasforma i campi del tipo di gioco in regole semplici. Quando arriva un evento, il servizio decide se si tratta di inizio, punto del primo partecipante, punto del secondo partecipante, fine oppure evento generico configurato su un sensore.

Il campo `value` permette di aggiungere piu di un punto. Per il calciobalilla vale normalmente 1, per le bocce puo valere da 1 a 3 e per le freccette puo valere 20, 50 o 60.

### 8.2 Fine automatica

Dopo un evento di punteggio il servizio confronta `score1` e `score2` con `score_limit`. Quando il limite viene raggiunto salva il vincitore, chiude la partita, rimette online il gioco, invia il comando agli attuatori e collega la partita a un torneo attivo compatibile.

### 8.3 Deduplicazione

MQTT QoS 1 puo consegnare lo stesso messaggio piu volte. `event_uuid` e univoco nel database. Se il servizio riceve di nuovo lo stesso UUID restituisce la partita senza modificare il punteggio.

### 8.4 Heartbeat

L edge invia un heartbeat ogni cinque secondi. Il Match Service aggiorna stato, `last_seen` e `last_sync`. Un controllo periodico segna offline i dispositivi silenziosi da piu di venti secondi.

### 8.5 Classi principali

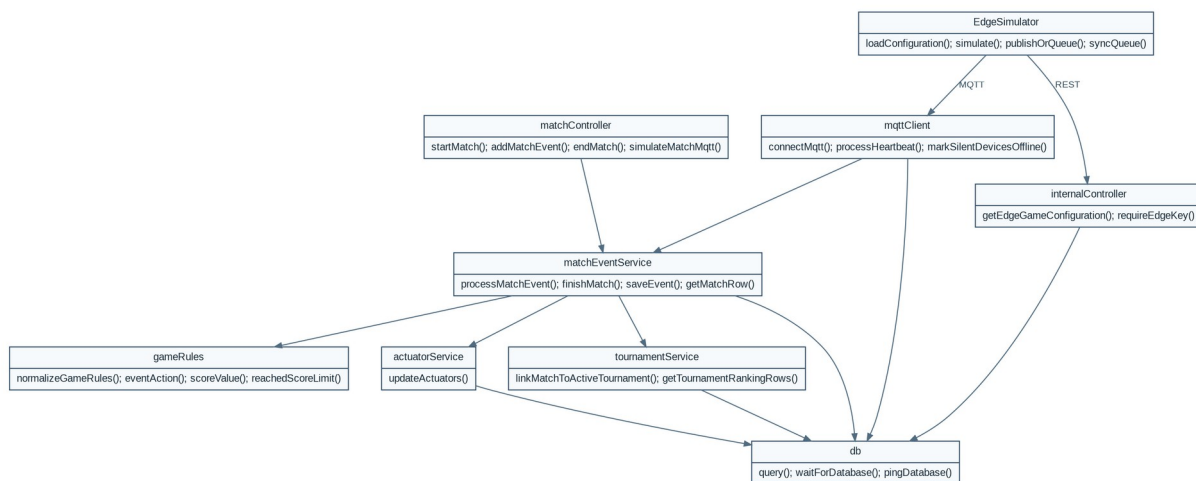


Figura 6 - Classi e moduli principali



## 8.6 Sequenza online

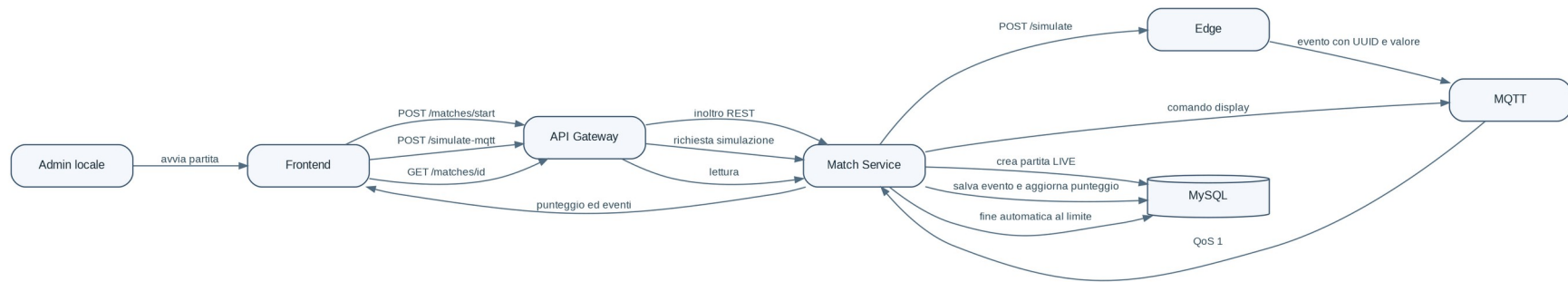


Figura 7 - Partita online

## 8.7 Sequenza offline

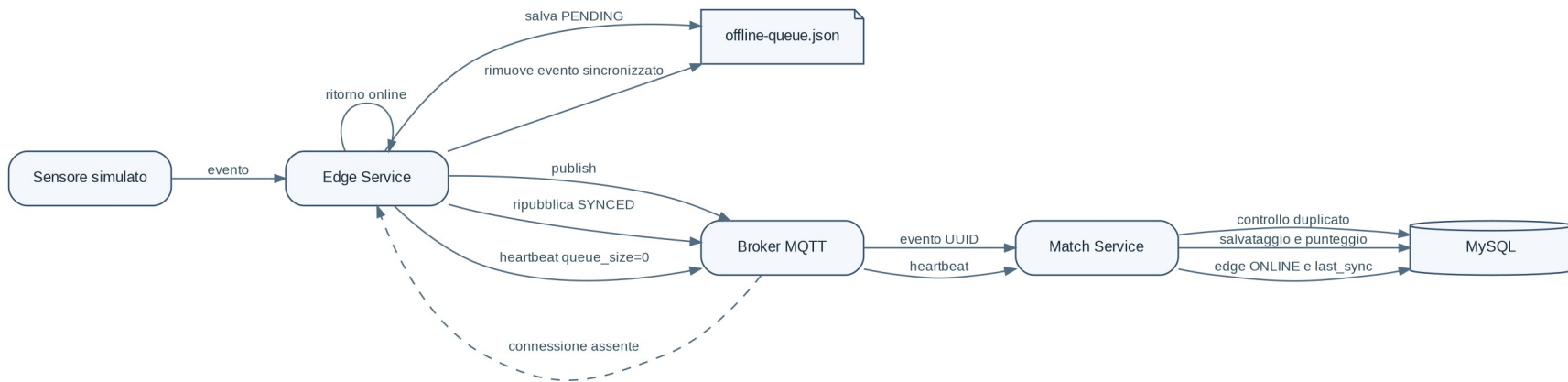


Figura 8 - Coda e sincronizzazione

## 9. Interfaccia utente

Il frontend usa pagine separate per i quattro ruoli. Tutte le pagine condividono barra laterale, intestazione, badge degli stati, tabelle e schede. La pagina della partita live aggiorna i dati ogni due secondi e mostra il valore dell'ultimo evento.

L'amministratore gioco può creare e modificare i tipi, cambiare gli eventi e aggiungere modelli sensore. L'amministratore locale configura edge, sensori e attuatori e avvia le partite.

## 10. Validazione

| Controllo                           | Risultato                              |
|-------------------------------------|----------------------------------------|
| Test unitari Node.js                | 29 superati, 0 falliti                 |
| Controllo sintassi                  | superato                               |
| Controllo struttura e link frontend | 174 verifiche superate                 |
| OpenAPI                             | 3.0.3 - 41 percorsi                    |
| Test end-to-end                     | script completo da eseguire con Docker |

Lo script di integrazione prova un flusso completo con regole personalizzate e un flusso offline delle freccette.

```
cd backend
npm test
npm run check
cd ..
node scripts/validate-project.js
node scripts/integration-test.js
```

## 11. Procedura di demo

1. Avviare il progetto con start-demo oppure docker compose up --build -d.
2. Mostrare i quattro ruoli e le relative dashboard.
3. Aprire il tipo Freccette e mostrare eventi e limite 301.
4. Avviare una partita e usare Simula con MQTT.
5. Aprire l'edge sulla porta 8090 e passare offline.
6. Avviare una nuova simulazione, mostrare la coda e tornare online.
7. Mostrare la partita terminata, gli attuatori, il torneo e le statistiche.

## 12. Limiti e sviluppi futuri

Il prototipo usa account demo e password in chiaro. In produzione servirebbero hash, token, HTTPS e una gestione più completa delle autorizzazioni tecniche. I sensori sono simulati e possono essere sostituiti con dispositivi reali mantenendo lo stesso payload MQTT.

Come sviluppo futuro si possono aggiungere prenotazioni, notifiche, tabelloni automatici, dati specifici per Monopoli e posizione delle bocce.

## Conclusione

PlayConnect realizza il flusso dal sensore alla statistica e continua a raccogliere dati durante una breve assenza di connessione. Le regole configurabili permettono di aggiungere altri giochi senza inserire nuovi nomi di evento nel Match Service.